# Explainable AI for Effective Malware Detection and Triage

Ananta Prasad Gaire, Aleksandar Shurbevski, Shozo Naito
Kyoto College of Graduate Studies in Informatics (KCGI)
Kyoto, Japan
Emails: st052439@m2.kcg.edu, a_shurbevski@kcg.edu, s_naito@kcg.edu

***Abstract*—Security Operations Centers (SOCs) increasingly rely on machine learning to triage large volumes of potentially malicious artifacts, but high-performing models are often difficult to trust because their predictions are opaque. This paper presents an Explainable AI (XAI) pipeline for static malware detection from Windows Portable Executable (PE) files that combines strong classification performance with analyst-oriented evidence. The method uses EMBER2024-derived tabular PE features and section-level structural representations rendered as compact "section images," then fuses both modalities for malware/benign classification. On the held-out test set, the fusion model achieves Accuracy = 0.9406 and malware F1 = 0.9392 at the default threshold, outperforming a tabular-only baseline. To improve decision transparency, the pipeline integrates Integrated Gradients (IG) for global and per-file tabular attributions and Grad-CAM for structural localization on section images. Results are reported under default and tuned operating thresholds to reflect precision–recall trade-offs relevant to deployment. Overall, the study shows that high-performing static malware detection can be paired with interpretable evidence artifacts—calibrated scores, ranked feature attributions, and heatmaps—that support defensible SOC triage.**

## I. Introduction

### A. Background and Motivation

Modern organizations continuously receive, generate, and execute large volumes of digital artifacts through routine workflows such as software distribution, web browsing, and endpoint updates. In these workflows, Windows Portable Executable (PE) files are especially important from a defensive standpoint because PE is the native executable structure of Windows systems. PE files remain a dominant malware delivery format and can be distributed through common channels such as downloads, attachments, or bundled installers. At Security Operations Center (SOCs) scale, the challenge is not only detecting threats but doing so at a speed and volume that manual inspection cannot match. To handle this scale, SOCs increasingly deploy [**?**] machine learning (ML) systems to automate triage and prioritize suspicious files for analyst review. Deep learning models are attractive in this setting because they can learn complex interactions in high-dimensional static features that rule-based filters and signature matching often miss [**?**]. Yet, cybersecurity operations is a high-stakes environment: model outputs often influence decisions such as quarantining endpoints, blocking execution, or escalating incidents. These actions require accountability and evidence. If a model behaves as a black box and only provides a label (e.g., malware/benign), analysts face a trust gap: either they accept a decision they cannot justify, or they must re-investigate alerts from scratch, which reduces the operational value of automation. This trust gap becomes even more critical as threats evolve and adversaries intentionally manipulate surface cues to evade detection.

### B. Explainable AI and Why It Matters in Cybersecurity

Explainable AI (XAI) refers to a family of methods and system designs intended to make the behavior of machine-learning models understandable to humans. In practical terms, XAI aims to answer questions such as: Which inputs contributed most to this prediction? What evidence increased or decreased the model's confidence? and What parts of the input were most influential? In this work, explainability is treated as evidence generation: explanations are produced as structured artifacts that accompany predictions and support verification, reporting, and analyst reasoning. This project adopts widely used attribution-based XAI methods because they can be applied directly to trained models without redesigning the model architecture. For tabular feature inputs and other structured representations, additive feature attribution approaches such as SHapley Additive exPlanations (SHAP) [**?**] provide local explanations (why one sample was classified in a certain way) and global summaries (which features are broadly important across many samples). For deep networks, Integrated Gradients [**?**] provides principled attributions by accumulating gradients from a baseline input to the actual input, offering a model-sensitive measure of feature influence. For CNN-based components, Grad-CAM produces a saliency heatmap that highlights spatial regions most responsible for a prediction, enabling a visual form of evidence over image-like representations [**?**]. These methods are well-established and widely reused, making them suitable for a rigorous pipeline in which both performance and interpretability are reported in a structured way. Explainability matters in cybersecurity for three reasons. First, it supports trust calibration. A high confidence score is not automatically trustworthy; analysts need to know whether the evidence aligns with known malicious patterns and whether the model is relying on meaningful signals. Second, explainability supports investigation efficiency. If an alert comes with evidence such as top contributing PE features and highlighted structural regions, analysts can validate or dismiss predictions faster,

which improves SOC throughput. Third, explainability helps reveal failure modes, including shortcut learning, spurious correlations, and inconsistent preprocessing. In security, these failure modes are not just academic; they can be exploited by attackers or can produce operational harm through excessive false positives. Because of this, explanations must be treated as outputs that require validation and careful interpretation. This is especially relevant for any saliency signal; prior work cautions that explanation outputs are not automatically faithful and must be evaluated rather than assumed [**?**].

### C. Research Aim and Scope

This research aims to develop and evaluate an explainable, proposal-aligned pipeline for static malware detection from Windows PE artifacts. The project is designed so that the pipeline produces both (a) strong predictive performance and (b) analyst-oriented evidence describing model reasoning. The work focuses on static detection using the EMBER2024 PE feature set [**?**] and a section-level structural representation derived from executables. The scope does not include dynamic malware execution traces, malware family attribution, network intrusion detection, URL crawling, attachment sandboxing, sender reputation systems, or production SOC deployment engineering.

### D. Malware Detection Branch: Design and Motivation

The malware branch investigates explainable static malware detection using the EMBER2024 PE feature representation [**?**]. Structurally, this representation maps each Portable Executable (PE) file into a fixed-dimensional vector of engineered features, summarizing byte-level statistics, header metadata, and import-related information. The dataset itself directly provides the first modality: a set of tabular descriptors that capture statistical properties of the file. However, tabular summaries can obscure section-level structural layout information. To address this, we introduce original descriptors as a second modality by rendering the executable's section layout into a compact image and processing it with a Convolutional Neural Network (CNN). This allows the system to learn structural patterns that the pre-calculated dataset features may miss. This choice reflects a practical intuition: PE files carry both statistical feature evidence (captured by engineered vectors) and structural evidence (captured by section organization), and combining both may strengthen classification while enabling richer explanations. To evaluate this hypothesis, we train two models: a TabOnly baseline that learns only from the aligned tabular feature vector, and a Fusion Tab+CNN model that merges a feed-forward tabular encoder with a lightweight CNN encoder over section images. On the held-out test split, the Fusion Tab+CNN model consistently outperformed the TabOnly baseline across both standard and tuned thresholds. The comparative results are summarized in Table 1.

We emphasize the tuned operating point because threshold selection is an intrinsic part of operational decision-making, while still reporting the standard threshold for transparency and comparability. Explainability is integrated in a modality-aligned manner. For tabular evidence, we use Integrated Gradients to compute both global attributions (aggregated evidence trends) and local attributions for individual files [**?**]. For the CNN branch, we use Grad-CAM [**?**] to generate heatmaps that highlight the section-image regions that most influence the malware decision. During development, we identify a critical reliability issue: when training-time feature normalization is missing during inference and explainability, predicted probabilities and explanations become inconsistent. By regenerating and enforcing the correct normalization statistics aligned to the enforced feature dimension, we restore probability fidelity and ensure attributions are computed on the same scale as training, which is necessary for meaningful comparison across cases.

TABLE I
PERFORMANCE COMPARISON ON EMBER2024 MALWARE DETECTION

| **Model** | $\tau$ | **ACC** | **F1** | **ROC-AUC** |
|---|---|---|---|---|
| Tab-only LGBM (EMBERv3) | 0.50 | 0.9794 | 0.9792 | 0.9979 |
| Tab + ImgEmb LGBM | 0.50 | 0.9795 | 0.9793 | 0.9979 |
| Tuned Tab + ImgEmb LGBM | 0.35 | 0.9802 | 0.9801 | 0.9979 |

### E. Contributions

The main contributions are:

- An explainable multimodal static malware detection pipeline using EMBER2024 PE features, including a TabOnly baseline and a Tab+CNN fusion model evaluated under both standard and tuned thresholds [**?**].
- A multimodal explanation strategy for malware analysis, combining Integrated Gradients for tabular attributions and Grad-CAM for section-image localization [**?**], [**?**].
- Reliability-focused implementation findings, including strict normalization alignment and feature-schema consistency for stable malware explanations.

## II. RELATED WORK

Static malware detection sits at the front line [**?**] of Security Operations Center (SOC) triage because it can be executed quickly, safely, and at scale, often before deeper investigation is feasible [**?**]. For Windows endpoints, a large share of malicious delivery still arrives as Portable Executable (PE) files [**?**] distributed through downloads [1], attachments, and staged droppers. The operational constraint is clear: defenders must filter high-volume file streams with limited analyst time, and the cost of errors is real. This is where the "trust gap" of modern machine learning becomes visible. Deep models can achieve strong predictive metrics, but SOC decisions still require justification [**?**]: why a file was quarantined and what evidence supports that action. Explainable AI (XAI) addresses this gap by producing interpretable signals (feature attributions and saliency maps) that connect model outputs to concrete input evidence. In cybersecurity, this evidence is not merely descriptive—it supports analyst validation, accelerates investigation, and reduces the risk of deploying opaque automation in high-stakes workflows. For this reason, this research treats

explainability as a first-class system output and evaluates XAI behavior as part of the overall malware detection pipeline [?], [?].

### A. Static Malware Detection

*1) Feature-engineered PE malware detection and the EMBER line of work:* A major practical tradition [?] in static malware detection is to extract specific, pre-defined properties from the Portable Executable (PE) file to create a fixed-length input vector. In the literature, these inputs are variously referred to as "handcrafted features" [?], [?], "engineered features" [?], or "tabular descriptors" [?]. To maintain consistency and precision, this paper refers to them as engineered features. These features typically include [?] byte-level statistics (e.g., histograms), entropy-related measures that can correlate with packing/obfuscation, metadata derived from headers/sections, and import/export information that reflects program behavior at a coarse structural level. The key strength of this approach is operational scalability: feature extraction and inference are fast [?], and fixed-dimensional vectors support repeatable benchmarking [?], thresholding, and controlled ablation studies [?]. The EMBER [?] dataset played a foundational role in making large-scale, reproducible PE-feature research possible by providing standardized feature extraction and evaluation conventions. EMBER2024 [?] extends this tradition by providing a large-scale, pre-computed dataset where each file is represented not as raw bytes, but as a vector of engineered values summarizing specific file characteristics. These features explicitly include: Byte-level statistics: Histograms and entropy calculations that capture the distribution of raw byte values. Header information: Metadata extracted from the COFF and Optional headers (e.g., linker versions, timestamps, and machine types). Section characteristics: Statistics regarding the size, entropy, and virtual memory layout of the executable's sections. Import and Export information: Hashed representations of imported libraries, imported functions, and exported symbols, which serve as proxies for the program's capabilities. By relying on these pre-calculated engineered features, the EMBER2024 representation enables scalable training and systematic evaluation at the feature level [?], [?]. Benchmark baselines and published results. To support reproducible comparison, the EMBER2024 authors also provide official benchmark models and runnable training/evaluation scripts in the public repository (see the examples/ directory) [?], [2]. These baselines are primarily LightGBM classifiers trained on EMBER feature version 3 vectors, intended to serve as reference performance levels for future work. For malicious/benign detection, the benchmark LightGBM classifier achieves ROC AUC = 0.9949 and reaches a true positive rate of 94.48% at a 1% false positive rate on the EMBER2024 test set [?]. For malware family classification, the benchmark LightGBM model reports accuracy = 67.97%, with weighted-F1 = 0.6664 and macro-F1 = 0.4371, reflecting the long-tail difficulty across many smaller families [?]. These published baseline results provide a consistent reference point that later experimental results (including ours) should be compared against.

*2) Representation learning from raw bytes and the "end-to-end" alternative:* In parallel to engineered-feature pipelines, a competing research direction [?], [?] argues that end-to-end models operating directly on raw bytes can discover discriminative patterns without relying on explicit feature design. MalConv [?] is a widely cited example of this idea [?], [?], demonstrating that convolutional architectures can ingest long byte sequences to classify executables. This family of approaches has expanded the field's understanding [?], [?] of representation learning for malware, but it also highlights practical constraints that are important for SOC-facing systems: very long sequences increase compute and memory costs [?], truncation or padding choices can affect what the model learns [?], [?], and mapping byte-level signals to analyst-friendly evidence can be challenging compared to structured PE features [?], [?]. These trade-offs matter to our project design. We stay within a controlled, fixed-dimensional representation (tabular PE vectors, plus a compact "section image" representation) to keep the pipeline scalable and explanation outputs interpretable. In other words, rather than choosing between "feature engineering" and "end-to-end bytes," our work aligns with literature [?], [?] that treats representation choice as an operational design decision: maintain tractable inputs that can support stable training, controlled evaluation, and explanation artifacts that analysts can act on.

*3) Malware-as-image and structural visual encodings:* Another influential branch [?], [?] of malware research transforms executables into image-like representations to enable CNN-based modeling. Early work [?] showed that visualizing binaries as images can support classification by capturing texture-like patterns that emerge from byte structure. More recent work [?] continues to explore image conversion techniquesnsometimes combined with additional features—showing that visual encodings can complement other representations when designed carefully. The literature further establishes that image-based approaches offer distinct advantages for interpretability and fusion. Research indicates that unlike purely tabular data, visual representations enable spatial explanation methods (e.g., Grad-CAM) that can highlight exactly which regions of a file structure influence the model's decision [?], [?]. Furthermore, studies in multimodal learning suggest that combining these structural visual signals with engineered feature vectors creates a more robust detection system, as the two modalities capture complementary patterns statistical evidence from features and spatial evidence from structure [?], [?].

*4) Multimodal and fusion modeling for static malware detection:* The literature increasingly suggests [?], [?] that no single static view fully captures malware behavior, especially under packing, obfuscation, and family diversity. As a result, fusion architectures that combine complementary signals have become more common. For example, research that jointly analyzes PE structure with entropy-related signals reflects the broader idea that combining structural and distributional

properties can improve malware classification robustness [3]. More generally, fusion is attractive because it provides redundancy: when one modality is less informative for a sample (e.g., a heavily packed file), another modality may still retain useful evidence. This trajectory in malware research supports the hypothesis that fusing structured PE vectors with compact structural visual representations can improve detection performance without abandoning interpretability [**?**], [3]. By comparing tabular-only baselines against fusion architectures, recent studies aim to quantify exactly how much robustness is gained by integrating these complementary static views.

*5) Explainability for malware: attribution for tabular signals and localization for CNN signals:* As deep learning becomes common in malware detection [**?**], explainability has shifted from "optional" to "required" for analyst-facing systems [**?**]. Two families of methods align naturally with multimodal designs: Attribution methods for tabular deep models. Integrated Gradients (IG) provides per-feature attributions for differentiable models and is widely used to interpret deep networks by measuring how input features contribute to predictions along a path from a baseline to the actual input [**?**]. For malware PE vectors, IG produces ranked feature attributions that can be summarized into analyst-readable tables (global importance trends and local evidence for a single file). Localization methods for CNN components. Grad-CAM produces class-discriminative heatmaps for CNN-based models, highlighting which spatial regions in an image-like input most influenced the classification [**?**]. For section-image encodings, Grad-CAM provides a direct visual "where the model looked" signal that can be inspected alongside the predicted probability. A critical point in XAI literature [**?**], [**?**] especially relevant to security is that explanation outputs must be reliable, not merely visually appealing. Sanity-check research shows that some saliency/explanation methods can fail in subtle ways (e.g., remaining similar even when model parameters or labels are randomized), motivating validation procedures rather than assuming explanation faithfulness by default [**?**]. This supports the methodological choice to treat explanation generation and validation as part of the system design, not a post-hoc screenshot [**?**], [**?**], [**?**].

### B. Summary and Positioning

The malware literature shows a consistent trajectory: high-performing models increasingly rely on deep learning and richer representations, but operational adoption in cybersecurity still requires transparent evidence and explanation reliability. EMBER-style PE feature benchmarks and multimodal fusion trends motivate our TabOnly baseline and Tab+CNN fusion design, while Grad-CAM and IG provide explanation modalities aligned to each input stream [**?**], [**?**], [**?**], [**?**], [**?**], [3]. This section establishes the research foundation for our implementation choices: controlled, scalable representations; fusion for complementary evidence; and explanations treated as measurable outputs that support analyst-oriented interpretation rather than black-box automation [**?**], [**?**], [**?**], [**?**], [3].

## III. Methodology

This paper proposes and evaluates an Explainable AI (XAI) framework for static malware detection from Portable Executable (PE) artifacts. The methodological design reflects common Security Operations Center (SOC) requirements where automated detection must operate under high volume and time constraints, while still producing evidence that supports analyst validation and reporting. In this context, the project treats explainability as a core system output rather than a post-hoc visualization, because security decisions often require justification and traceability. The methodology emphasizes scalability and multimodal evidence, using large-scale PE feature vectors and section-derived structural representations to study whether complementary static views improve robustness. Evaluation is conducted under a consistent protocol and includes probability threshold analysis to reflect deployment-style trade-offs in security settings.

### A. Malware Detection

*1) Data Source and Dataset Representation:* The malware branch is implemented as a static detection problem using an EMBER2024-derived PE feature representation [**?**], grounded in the broader EMBER benchmark tradition of large-scale, reproducible PE malware research [**?**]. Each executable is represented as a fixed-dimensional feature vector extracted from intrinsic properties of the PE artifact, and each sample is assigned a binary label indicating benign or malicious class membership. This representation is suitable for security triage because it supports efficient batch inference and allows structured explanation outputs at the feature level. To support large-scale processing, the dataset is organized in a sharded format where each shard stores a feature matrix and corresponding labels. Formally, each shard contains a feature matrix and a label vector, enabling streaming-style loading and memory-safe inference. This structure is important because the dataset scale requires repeated passes for training, validation, testing, and explainability generation without relying on full in-memory loading. A key implementation detail in this project is strict control of the effective feature dimension used by trained models and explainability routines. While the raw metadata indicates a slightly larger nominal feature dimension, the final training and evaluation enforce a consistent dimension of 515 features to ensure checkpoint compatibility and explanation comparability. This alignment step is necessary because neural network layers are parameterized for a specific input dimensionality, and even a single-feature mismatch prevents strict model reuse and can lead to inconsistent explanations.

*2) Preprocessing and Normalization:* Feature scaling is treated as a first-order methodological requirement because deep models trained on heterogeneous feature groups can become unstable or misleading if preprocessing differs between training, inference, and XAI computation. In this project, feature normalization statistics are computed on the training split and reused consistently in all later steps. Let and denote the per-feature mean and standard deviation estimated from training data. For each enforced input vector, the normalized

vector is computed as: where is a small constant to prevent division-by-zero. This project explicitly enforces preprocessing consistency because explanation reliability depends on the model operating in the same numerical feature space that it was trained on. Inconsistent normalization can change predicted probabilities and distort gradient-based explanations, which undermines the central objective of producing analyst-usable evidence.

*3) Model Architectures:* To evaluate the benefit of multimodal evidence while retaining interpretability, the project implements two complementary architectures: a tabular baseline and a multimodal fusion model. Both operate on the same normalized tabular features, and the fusion model additionally incorporates a compact structural representation processed by a CNN. Tabular-only neural classifier (TabOnly) The baseline model is a deep tabular classifier that maps normalized PE feature vectors to class logits. Let denote the network with parameters, and let be the output logits for the benign and malware classes. The malware probability is computed using the softmax mapping: This baseline quantifies how much discriminative power is available from tabular PE features alone under the project's normalization and evaluation protocol. Multimodal fusion classifier (Tab+CNN) The fusion model integrates two encoders: a tabular encoder over PE features and a CNN encoder over section-derived structural representations. For each sample, the model processes the pair, where is the normalized tabular vector and is the corresponding section-derived representation. The tabular branch produces an embedding, and the CNN branch produces an embedding. These embeddings are fused (via concatenation) and passed through a decision head to produce logits and probabilities. This design is consistent with the broader motivation for combining complementary static views: PE features capture global structural/statistical signals, while the CNN branch can capture localized structural patterns present in the section-derived representation. Importantly for this research, the architecture is also designed to support modality-aligned explanations: feature attributions for the tabular branch and spatial saliency for the CNN branch [**?**], [**?**].

*4) Evaluation Protocol and Operational Thresholding:* Evaluation is performed on a held-out test split that contains 960,000 samples in the finalized experiment configuration. For each test sample, the model outputs a malware probability. A decision threshold converts probability into a binary prediction: The project reports performance under the conventional operating point and additionally performs a threshold sweep to select an operating threshold optimized for the malicious class F1 score. This selection is formalized as: The purpose of this step is not to "inflate" metrics but to reflect how SOC deployments often require tuned decision boundaries depending on tolerance for false positives versus missed detections. The methodology therefore preserves transparency by reporting both the default threshold and the best-F1 threshold.

*5) Explainability Methods (Malware Branch):* The malware branch integrates explainability as an explicit pipeline stage so that each prediction can be accompanied by interpretable evidence. The approach uses Integrated Gradients for tabular attribution and Grad-CAM for CNN-based spatial localization, which are widely used explanation methods aligned to differentiable networks and convolutional representations [**?**], [**?**]. Integrated Gradients for tabular feature attribution Integrated Gradients (IG) attributes a model score to individual input features by integrating gradients along a path from a baseline to the input [**?**]. For feature, the IG attribution is: In this project, IG is operationalized in two modes. First, global explanations are obtained by aggregating mean absolute attributions across a background subset of samples, producing a stable summary of which PE features consistently influence predictions. Second, local explanations are produced for selected test samples, yielding ranked feature evidence suitable for analyst-style review. Grad-CAM for CNN branch localization Grad-CAM produces a class-discriminative heatmap over a convolutional representation by weighting activation maps using the gradients of the target score [**?**]. Given feature maps, Grad-CAM computes weights and forms a heatmap: The resulting saliency map highlights which spatial regions in the section-derived representation contribute most strongly to the malware decision. In the fusion setting, Grad-CAM complements IG by providing structural evidence tied to the CNN pathway.

*6) Implementation and Reproducibility Workflow:* To ensure the experiments are repeatable and auditable, the malware pipeline follows a structured execution workflow. First, the dataset is loaded in a sharded manner to support large-scale iteration without exhausting memory, and feature dimensionality is strictly aligned to the trained model configuration. Second, training-derived normalization statistics are computed once and reused consistently for all downstream stages, including test evaluation and gradient-based explanations. Third, the trained model states are saved at convergence and reused for evaluation and explainability generation, ensuring that reported metrics and explanations are derived from the same model parameters. Finally, the pipeline produces traceable outputs that link each evaluated sample (or chosen analysis subset) to its predicted probability and explanation artifacts, enabling verification and report assembly without requiring ad-hoc recomputation. This workflow is especially important in XAI settings because explanation methods can be sensitive to model state, preprocessing, and numerical scale. By keeping the preprocessing statistics and inference configuration consistent across evaluation and explanation stages, the project preserves fidelity between model behavior and explanation outputs.

## IV. Implementation and Results

### A. Malware Detection: Implementation and Results

*1) System Implementation Overview:* For malware detection, we implemented two deep-learning classifiers operating on engineered static PE feature representations. The baseline model (TabOnly) consumes a fixed-length numeric feature vector and outputs a malware probability. The main model (Fusion Tab+CNN) augments the same tabular evidence with a second modality: a compact section-structure representation

processed by a CNN, followed by feature fusion and a final decision head. The design goal is practical: keep inference scalable for millions of files while producing analyst-facing evidence rather than a single opaque score.

*2) Decision Rule, Threshold Sweep, and Reported Metrics:* For each test sample i, the model outputs a malware probability $p_i = p(y = 1|x_i)$. Given a decision threshold $\tau$, the predicted label is $\widehat{y}_i(\tau) = 1$ if $p_i \geq \tau$ and 0 otherwise. From the resulting confusion matrix counts, we compute Accuracy, Precision (malicious), Recall (malicious), and F1 (malicious). Because operational malware detection often requires balancing false negatives (missed malware) against false positives (benign files escalated), we report results at the conventional $\tau = 0.50$ and also report the best-F1 operating point identified by sweeping $\tau$ over multiple values.

*3) Final Test Performance:* Table 4.1 summarizes the final test performance for TabOnly (baseline) and Fusion Tab+CNN (main). Fusion is reported as the primary outcome, while TabOnly provides a reference point for quantifying the benefit of multimodal fusion. Comparison to official EMBER2024 baselines. To contextualize the results in Table 4.1, I compare them with the official EMBER2024 benchmark models reported by Joyce et al. and distributed with example scripts [1, 37]. The published baseline for malicious/benign detection is reported using threshold-independent operating metrics (ROC AUC = 0.9949 and TPR = 94.48% at 1% FPR) [1]. While reporting in this paper focuses on Accuracy/F1 at selected operating thresholds ($\tau = 0.50$ and best-F1), this comparison remains useful: the EMBER2024 benchmark demonstrates that engineered PE features can yield strong separability on the test distribution, and the TabOnly and Fusion models aim to achieve comparable operational performance while additionally producing analyst-oriented explanations (Integrated Gradients for tabular evidence and Grad-CAM for structural section images).

TABLE II
PERFORMANCE COMPARISON OF MALWARE DETECTION RESULTS ON THE FULL TEST SET (N=960,000).

| Model | $\tau$ | AUC | ACC | Precision |
|---|---|---|---|---|
| Tab-only LGBM (EMBERv3 2568) | 0.50 | 0.997938 | 0.979406 | 0.9877 |
| Tab+ImgEmb LGBM (2568+128) | 0.50 | 0.997949 | 0.979483 | 0.9875 |
| Tab+ImgEmb LGBM (tuned) | 0.35 | 0.997949 | — | — |

Malware detection performance at $\tau = 0.50$ and at the best-F1 threshold from the sweep.

The main takeaway is that fusion improves recall and F1, meaning it catches more malicious samples while maintaining strong precision. Malware performance comparison (Accuracy, F1, Recall) for TabOnly and Fusion at two operating points.

*4) Confusion Matrices and Operational Interpretation:* Confusion matrices translate abstract metrics into tangible operational costs. In the context of malware triage, False Negatives (FN) correspond to malware that evades blocking and reaches endpoints (risk of compromise), while False Positives (FP) correspond to benign files that are incorrectly quarantined or escalated (business disruption). At the standard threshold,

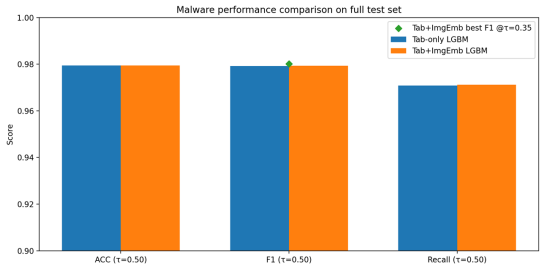


Fig. 1. Malware performance comparison (Accuracy, F1, Recall) for TabOnly and Fusion at two operating points.

the Fusion model substantially reduces the number of missed malware samples compared to the TabOnly baseline. This reduction represents a meaningful security improvement at the default operating point, as it directly lowers the probability of infection without relying on aggressive threshold tuning.

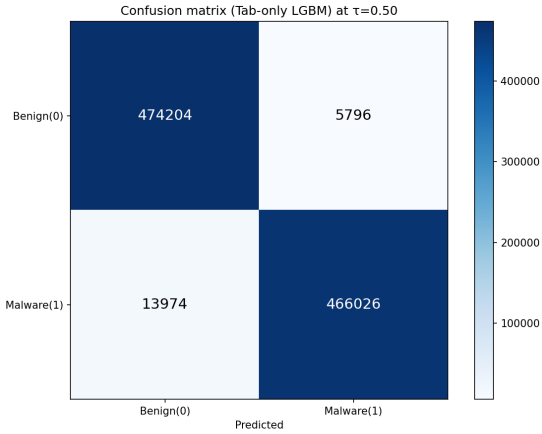


Fig. 2. Confusion matrix (Tab-only LGBM) at $\tau = 0.50$

TabOnly confusion matrix at $\tau = 0.50$.

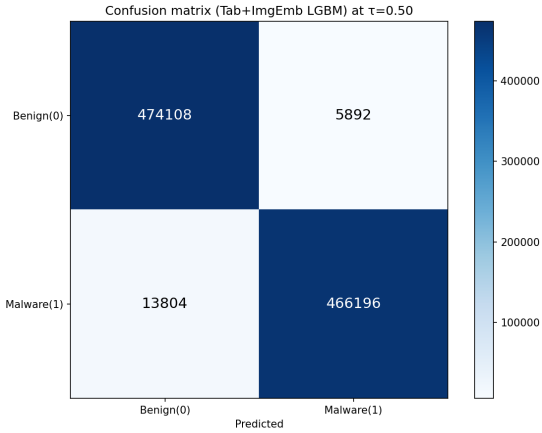


Fig. 3. Confusion matrix (Tab+ImgEmb LGBM) at $\tau = 0.50$

Fusion Tab+CNN confusion matrix at $\tau = 0.50$.

*5) Explainability Report for Malware: Integrated Gradients and Grad-CAM:* To make malware decisions reviewable, we generate explanations that align with the multimodal architecture. For the tabular branch, Integrated Gradients (IG) produces a signed attribution score per feature: positive values push the model toward malware, negative values push it toward benign. For the CNN branch, Grad-CAM produces a heatmap over the section-structure representation, indicating which regions contributed most to the malware class score. Together, these outputs provide two complementary forms of evidence: 'which PE features mattered' and 'which structural regions mattered.' Global IG summary (model-wide feature importance) Global IG summarizes which features consistently influence predictions across many samples. Table 4.2 lists the top-ranked features by mean absolute IG magnitude. Even when feature identifiers are not semantically descriptive, this ranking is valuable for reproducibility and for later mapping to feature groups (e.g., byte statistics, entropy-related features, or section-derived indicators). Local IG + Grad-CAM: four representative case studies Local explanations are critical for incident response because analysts validate individual alerts. We therefore generated local IG attributions and Grad-CAM heatmaps for four representative test indices. These examples were selected to cover different decision regimes: low-confidence malware, high-confidence benign, high-confidence malware, and a borderline benign case near common operating thresholds. Table 4.3. Local explanation summary for four representative malware samples (Fusion model). POS increases malware probability; NEG decreases it. A useful detail for interpreting Table 4.3 is that IG explanations are additive and signed, so they naturally show both supporting and suppressing evidence. In other words, the model may find some features that look benign (negative attribution) even in a malware sample, but the final probability reflects the net effect of all features and the CNN branch. This push-pull behavior is particularly informative for borderline cases because it reveals why the model hesitated. Grad-CAM heatmaps (CNN branch evidence) Figures 4.6 through 4.9 present the Grad-CAM heatmaps for the corresponding samples. These visualizations indicate where the CNN branch identifies discriminative structural evidence within the section representation. In practice, these heatmaps serve as visual 'pointers' for analysts; they do not replace full reverse engineering but efficiently highlight which parts of the structural view were most influential in the model's decision.

*6) What the Malware Results Mean:* The malware results support three conclusions. First, Fusion Tab+CNN consistently outperforms the tabular baseline, indicating that the section-image branch contributes complementary information rather than redundant noise. Second, the improvement is most meaningful in recall: at $\tau = 0.50$, Fusion misses far fewer malicious samples than TabOnly, which is the more costly error type in many security contexts. Third, explainability artifacts help justify decisions in uncertain cases; for example, the borderline benign sample shows how positive (malware-leaning) and negative (benign-leaning) signals compete near the operating threshold, which is exactly where analyst review is most valuable.

## V. Discussion

This section interprets the implementation outcomes and empirical results, analyzing their broader implications for explainable cybersecurity threat detection. The discussion is organized around three primary analytical dimensions: Multimodal Improvement Analysis: An examination of why the multimodal malware fusion model outperformed the tabular baseline, focusing on the specific contributions of the structural view versus the engineered features. Limitations and Scope: A critical assessment of the specific limitations that define the boundary of the claims in this report, ensuring the validity of the findings is properly scoped. Where relevant, this section also provides evidence-based recommendations for operation and future extension.

### A. Key Findings

*1) Malware detection: key outcomes and why they matter:* In the malware component, we implemented two PE-based classifiers on the sharded EMBER2024 feature format: a tabular-only baseline (TabOnly, 515-dimensional after ablation) and a multimodal fusion model that combines the same tabular vector with a CNN representation of section-structure images (Fusion Tab+CNN). Both models output a probability score for the malicious class and a binary decision derived from a configurable threshold. The design goal was not only high accuracy but also explainability Integrated Gradients (IG) [**?**] was used to attribute the tabular contribution, and Grad-CAM [**?**] was used to localize the CNN evidence on the section-image representation. The evaluation results show a consistent advantage for multimodal fusion. At the standard 0.50 decision threshold, the tabular baseline achieved an accuracy of 0.9123 with malicious-class F1 of 0.9081 and malicious recall of 0.8666. The fusion model improved the same operating point to 0.9406 accuracy, 0.9392 malicious F1, and 0.9179 malicious recall. In operational terms, the most important improvement is the reduction in false negatives: the fusion model missed substantially fewer malicious files than the tabular-only model at the default threshold. In many cybersecurity settings, this is a high-impact gain, because a single missed malware sample can translate into downstream compromise risk, whereas a small increase in false positives can often be managed through triage policies.

*2) Why multimodal fusion improved malware detection:* The performance gap between TabOnly and Fusion is consistent with the idea that tabular PE features and section-structure representations encode complementary evidence. The tabular vector summarizes file properties as engineered statistics (e.g., byte-level distributions and other numeric descriptors). This representation is compact and highly informative, but it is also vulnerable to feature ambiguity: different malware families can share similar statistics, and sophisticated packing or obfuscation strategies can reduce the separability of purely engineered features. By contrast, the section-image branch

encodes a different view of the binary structural layout and relative patterns derived from section information. Even when a malware sample manipulates some global statistics, structural cues can remain informative (for example, abnormal section composition or atypical patterns in the section-derived representation). Fusion effectively behaves like an analyst combining multiple kinds of evidence: a numerical profile plus a structural profile. This reduces over-reliance on any single feature group and tends to improve recall in heterogeneous malware collections.

*3) Threshold behavior and deployment interpretation (malware):* A notable observation from the threshold sweep is that both models achieved their best malicious F1 at a lower threshold (0.35) than the default 0.50. For the fusion model, the best scanned point (0.35) reached 0.9420 accuracy with malicious precision of 0.9432, recall of 0.9406, and F1 of 0.9419. This confirms that the probability outputs are sufficiently calibrated for policy-driven tuning: by lowering the threshold, the system increases malware capture (recall) at the cost of higher false positives. Importantly, threshold selection is not only a technical choice; it is a risk-management policy. In a high security environment, a lower threshold may be justified to reduce missed malware, provided that the organization can handle additional triage. In environments with limited analyst capacity, a higher threshold may be preferable to reduce alert volume. The results in this report demonstrate that the fusion model supports this operational flexibility by offering a strong baseline at 0.50 and an improved balanced point at 0.35.

*4) Explainability contribution (malware): what IG and Grad-CAM add:* The explainability pipeline materially strengthens the malware component beyond raw metrics. IG provides directional attributions on the tabular features, which can be interpreted as evidence that pushes the model toward malicious or benign predictions. Grad-CAM, applied to the CNN branch, produces spatial localization maps on the section-image representation, showing where the fusion model concentrated its structural evidence. From a security workflow perspective, this pairing creates a two-view explanation: (i) which engineered features mattered most and whether they supported or suppressed the malicious decision (IG), and (ii) which structural regions the CNN considered influential (Grad-CAM). This is particularly valuable in borderline or error cases (e.g., low-confidence malware or near-threshold benign samples), because the explanations offer a rationale that can inform whether an analyst should quarantine, block, or escalate the file for deeper analysis.

### B. Limitations

*1) Malware detection limitations:* Dataset and labeling constraints. Static malware datasets often reflect labeling derived from antivirus consensus or detection heuristics rather than ground-truth behavioral verification. This introduces label noise risk, especially for borderline samples (potentially unwanted programs, grayware, or newly emerging families). As a result, performance metrics should be interpreted as effectiveness on the dataset's labeling standard rather than as an absolute guarantee of real-world correctness. Generalization and temporal drift. Malware evolves rapidly, and a model trained on a fixed historical distribution can degrade when deployed against new packing techniques, novel families, or shifting ecosystem trends. The reported test results are strong within the prepared split, but external validation on more recent or geographically distinct corpora would be required to claim robust long-term generalization. Feature schema sensitivity (516 $\rightarrow$ 515 ablation). The final evaluated checkpoints used a 515-dimensional input, while the raw shards were 516-dimensional. This introduced implementation complexity and makes the pipeline sensitive to feature schema changes. Although the project resolved this through consistent dimension forcing and matching normalization statistics, it highlights the need for strict feature versioning and careful checkpoint–schema alignment in any future extension. Explainability constraints (IG and Grad-CAM). IG attributions depend on the choice of baseline and background distribution and may vary across these choices. Grad-CAM provides localization of CNN influence but does not prove causality; it highlights correlated regions that influence activations, and results can depend on the chosen layer and upscaling method. Therefore, both explanation types should be treated as supportive evidence for analyst triage, not as definitive proof of maliciousness. Evaluation scope. The current evaluation focused on accuracy, precision, recall, F1, and threshold sweeps. In operational environments, additional diagnostics may be important, including PR-AUC/ROC-AUC, probability calibration, per-family performance breakdowns, robustness against packing/obfuscation, and concept-drift monitoring. These were outside the immediate implementation scope but are relevant for deployment-grade assessment.

### C. Lessons learned and practical recommendations

- **Use fusion as the primary detector.** The fusion model consistently improved recall and F1 while retaining strong precision, and it provides richer evidence through combined IG and Grad-CAM outputs.
- **Retain TabOnly as a baseline.** The tabular-only model remains useful for ablation comparison and for faster first-pass screening when compute is constrained.
- **Treat thresholding as policy.** Threshold choice can shift behavior from conservative blocking toward higher malware capture. In practice, define at least two operating modes: (i) conservative mode for low-noise environments and (ii) high-recall mode for high-security or quarantine/review workflows.
- **Improve semantic interpretability.** Mapping feature indices to descriptive PE semantics and standardizing explanation-card layouts can improve analyst comprehension and evidence usability.

### D. Discussion Summary

Overall, the malware results support the central claim that multimodal fusion (tabular PE features plus structural section

representation) improves detection quality over a tabular-only baseline, primarily by reducing missed malware while maintaining strong precision. At the same time, the pipeline has practical limitations—including dataset representativeness, generalization drift, feature-schema sensitivity, and method-specific explainability constraints—that define what can be confidently claimed and motivate the next engineering and research steps.

## VI. Conclusion and Future Work

### A. Conclusion

This paper demonstrates that explainable AI can be operationally meaningful for cybersecurity threat detection when detection performance, threshold calibration, and evidence-oriented explanations are developed together. The multimodal fusion model improved detection and reduced missed malware while producing interpretable evidence through feature attributions and structural localization. Operationally, the results show that threshold selection must be treated as a deployment policy rather than a fixed default, and that strict preprocessing/feature-schema alignment is essential for reproducible XAI outputs.

### B. Future Work

The future work listed below is intentionally constrained to extensions that follow directly from the implemented pipelines and the observed limitations. The aim is to improve interpretability depth, robustness to distribution shift, and deployment readiness without changing the project's core direction.

*1) Deepen semantic interpretability and robustness:* First, the tabular feature space should be mapped to richer semantic names so that attribution reports can be interpreted beyond index-based identifiers. Second, evaluation should be expanded to include probability calibration, precision–recall AUC, and per-family or time-split testing to better quantify concept drift and generalization to new malware. Finally, the multimodal model can be stress-tested with controlled ablations (tabular-only, image-only, fusion) and alternative fusion mechanisms to strengthen claims about which modality contributes most under different malware behaviors.

*2) Integrate explanations into an analyst-ready evidence card:* An immediate practical extension is to generate a unified analyst evidence card for each inspected executable. This card would consolidate the predicted probability, the thresholded decision, top positive and negative feature attributions (Integrated Gradients), and the corresponding structural localization (Grad-CAM) into one consistent format. A standardized card would improve usability for triage and facilitate case-based audit trails.

*3) System-level future work: monitoring and governance:* The system would also benefit from monitoring mechanisms that detect performance drift and explanation drift over time. In cybersecurity deployments, adversary behavior evolves rapidly; therefore, regular re-evaluation and documented model updates are required. A lightweight governance layer—tracking dataset versions, feature schemas, threshold settings, and explanation outputs—would make the overall system more trustworthy and maintainable.

### C. Closing Remarks

The proposed future work focuses on strengthening robustness, improving interpretability depth, and translating explanation artifacts into consistent analyst-ready outputs suitable for practical adoption.


## Acknowledgment

The authors thank Associate Professor Aleksandar Shurbevski for guidance and feedback throughout this project, and Professor Shozo Naito for sponsorship, resources, and continued support. The authors also thank the faculty and staff of Kyoto College of Graduate Studies for Informatics (KCGI) for providing a stimulating research environment.


## References


[1] Recorded Future, "H1 2025 malware and vulnerability trends," Recorded Future, Somerville, MA, USA, Tech. Rep., 2025. [Online]. Available: https://www.recordedfuture.com/research/h1-2025-malware-and-vulnerability-trends

[2] FutureComputing4AI, "Ember2024 (repository) — examples and benchmark scripts," GitHub. [Online]. Available: https://github.com/FutureComputing4AI/EMBER2024/tree/main/examples

[3] D. E. Simos, F. Zola, and F. Skopik, "Not on my watch: Ransomware detection through classification of high-entropy file segments," *Journal of Cybersecurity*, vol. 11, no. 1, p. tyaf009, 2025.